

Безопасность и криптографические методы в блокчейне

Изучение методов шифрования, таких как асимметричная криптография и хеширование

Базовые элементы криптографии

Системы и протоколы безопасности состоят из базовых криптографических элементов. В следующих разделах вы познакомитесь с алгоритмами шифрования, необходимыми для построения протоколов и систем безопасности. Протокол безопасности – это последовательность шагов, которые нужно предпринять для достижения необходимых стандартов безопасности, используя подходящие механизмы. В эксплуатации находятся различные виды таких протоколов – например, протоколы аутентификации, неотказуемости и управления ключами. Схема базовых элементов криптографии может быть визуализирована следующим образом (рис 48)

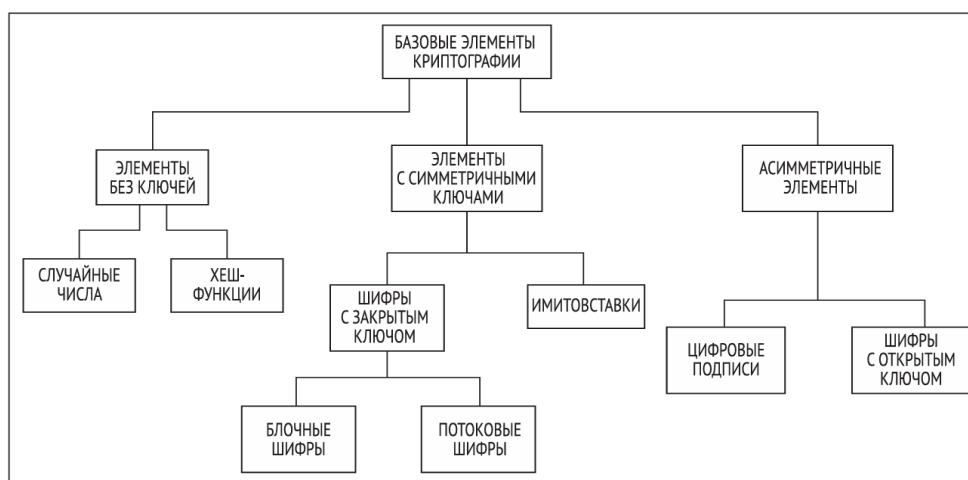


Рис 48 - Схема базовых элементов криптографии

Базовые элементы криптографии

Как видно на этой диаграмме, криптографию можно разделить на две основные категории: симметричную и асимметричную. Эти элементы будут рассмотрены далее.

Симметричная криптография

Симметричной называют криптографию, в которой для шифрования и дешифрования данных используется один и тот же ключ. Этот метод также известен как шифрование с общим ключом. Ключ должен быть выбран или согласован до того, как между сторонами коммуникации произойдет обмен информацией. По этой причине такой подход также называют шифрованием с закрытым ключом. Симметричные шифры бывают двух видов: потоковые и блочные. Типичными примерами блочных шифров являются стандарты DES (Data Encryption Standard) и AES (Advanced Encryption Standard); потоковые шифры часто применяются в стандартах RC4 и A5.

Потоковые шифры Потоковые шифры представляют собой криптографические алгоритмы, которые применяются к тексту побитово, используя ключевой поток. Существует два вида потоковых шифров: синхронные и асинхронные:

В синхронных потоковых шифрах ключевой поток зависит только от ключа.

В асинхронных потоковых шифрах ключевой поток также зависит от шифруемых данных.

В потоковых алгоритмах шифрованием и дешифрованием занимается одна и та же функция, поскольку они представляют собой такие простые операции, как сложение по модулю 2 (исключающее ИЛИ). Фундаментальным требованием, предъявляемым к потоковым шифрам, являются безопасность и случайность ключевых потоков. Чтобы этого добиться, в аппаратном виде реализуются генераторы случайных и псевдослучайных чисел, которые обязаны быть безопасными с точки зрения криптографии (рис 49)

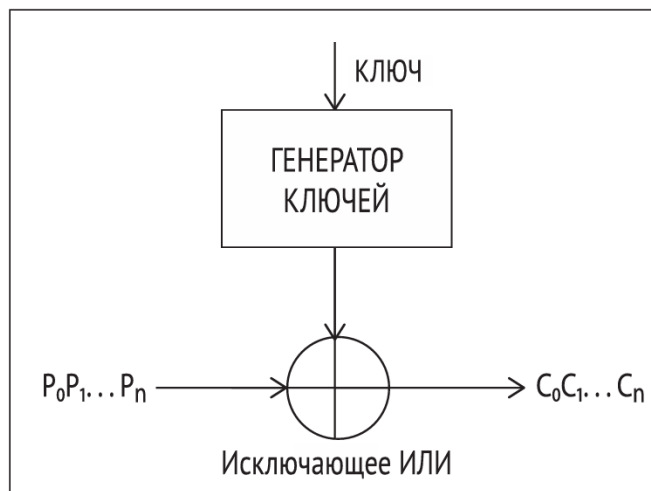


Рис 49 - Принцип работы потокового шифра

Блочные шифры Блочный шифр – это алгоритм шифрования, который разбивает шифруемый текст на блоки заданной длины и затем преобразует их один за другим. Блочные шифры обычно основаны на стратегии, известной как шифр Фейстеля. В современных блочных шифрах, таких как AES (Rijndael), применяется сочетание подстановки и перестановки, которое называют SP-сетью (англ. Substitution-Permutation Network – подстановочно-перестановочная сеть). Шифры Фейстеля основаны на структуре одноименной сети, разработанной Хорстом Фейстелем. Идея этой структуры состоит в многократном применении одних и тех же операций, что позволяет добиться таких криптографических свойств, как перемешивание и рассеивание. Сети Фейстеля работают путем разделения данных на два блока (левый и правый) и их последующей обработки с помощью циклических функций с ключом; после определенного количества итераций достигается достаточный уровень псевдослучайной перестановки. Перемешивание усложняет связь между исходным и зашифрованным текстами. Это обеспечивается благодаря подстановке. На практике это может выглядеть как замена буквы А в исходном тексте на букву Х в зашифрован ном результате. В современных криптографических алгоритмах подстановка производится с помощью таблиц замены, которые называются блоками подстановок. Благодаря рассеиванию исходный текст статистически «размывается» по итоговому результату. Это гарантирует, что при изменении хотя бы одного бита в исходном тексте зашифрованный результат в среднем меняется как минимум наполовину. Перемешивание нужно для того, чтобы найти ключ шифрования было крайне сложно, даже при наличии множества пар исходных и зашифрованных данных, в которых используется один и тот же ключ. На практике это достигается за счет перемещения или перестановки. Ключевым преимуществом использования шифра Фейстеля является то, что операции шифрования и дешифрования почти ничем не отличаются – нужно лишь поменять ход процесса на противоположный. Главным примером шифров на основе сетей Фейстеля является DES

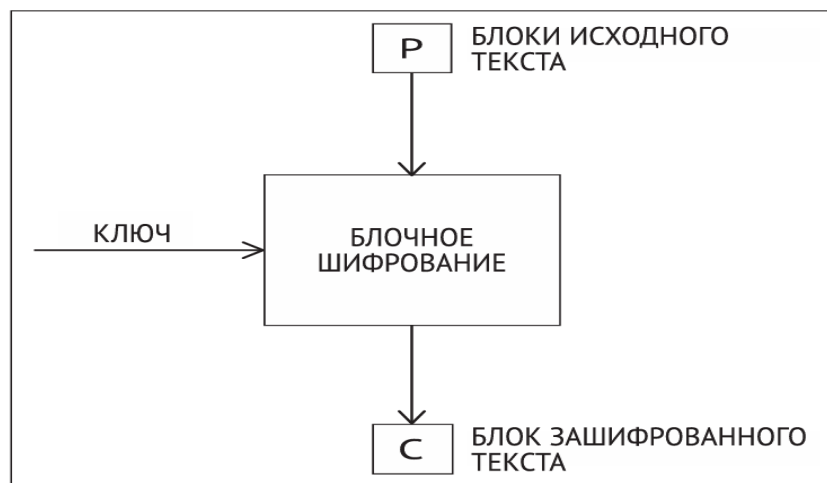


Рис 50 - примером шифров на основе сетей Фейстеля

Упрощенный пример блочного шифрования

Блочные шифры могут соответствовать таким моделям, как режим простой замены (англ. Electronic Code Book, или ECB), режим сцепления блоков шифротекста (англ. Cipher Block Chaining, или CBC), режим обратной связи по выходу (англ. Output Feedback, или OFB) и режим счетчика (англ. Counter mode, или CTR). Эти модели определяют способ применения функции к исходному тексту.

Блочный режим шифрования

В этом режиме исходный текст делится на блоки заданной длины в зависимости от типа используемого шифра. Затем к каждому блоку применяется функция шифрования. В следующих подразделах мы познакомимся с наиболее распространенными блочными режимами шифрования.

Режим простой замены (ECB)

Это базовый режим, в котором зашифрованные данные генерируются в результате последовательного применения алгоритма шифрования к каждому блоку исходного текста. Этот режим является наиболее простым, однако его не следует использовать на практике, так как он небезопасен и может раскрыть информацию.

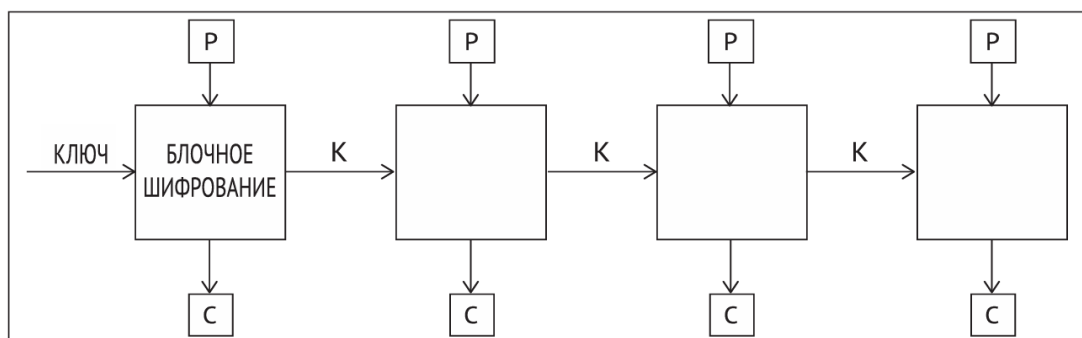


Рис 51 - Блочный режим шифрования

Режим простой замены для блочных шифров

На рис. 51 мы видим, что исходный текст P вместе с ключом подается на вход функции шифрования, а на выходе получается шифротекст C .

Режим сцепления блоков шифротекста (CBC) В этом режиме к каждому блоку исходного текста применяется исключающее ИЛИ с ранее зашифрованным блоком. Для шифрования первого блока CBC использует вектор инициализации (англ. Initialization vector, или IV), который рекомендуется выбирать случайным образом (рис. 51).

Режим сцепления блоков шифротекста

Режим счетчика (CTR) В этом режиме блочный шифр, по сути, используется как потоковый. Для генерации ключевого потока к значению счетчика добавляется случайное и уникальное число

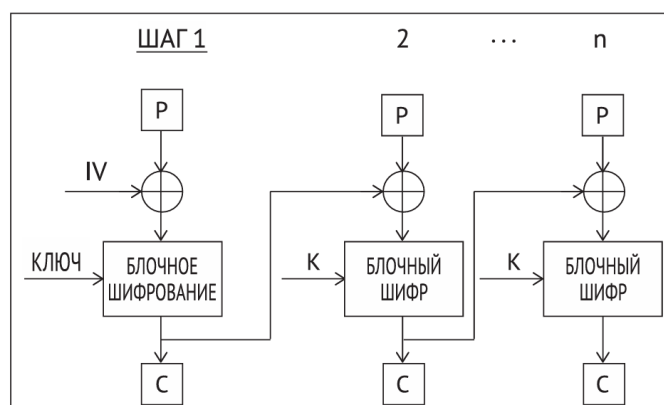


Рис 52 - Блочный режим шифрования

Режим счетчика

Существуют и другие режимы, которые применяются в различных ситуациях, например режим обратной связи по шифротексту (англ. Cipher Feedback или CFB), счетчик Галуа (англ. Galois Counter, или GCM) и режим обратной связи по выходу (англ. Output Feedback, или OFB).

Режим генерации ключевого потока

В этом режиме криптографическая функция генерирует ключевой поток, который затем применяется к исходному тексту с помощью исключающего ИЛИ, за счет чего и происходит шифрование.

Режим имитовставки

В этом режиме результатом функции шифрования является имитовставка – криптографическая контрольная сумма, которая обеспечивает целостность данных. Самый распространенный способ генерации имитовставок на основе блочных шифров называется СВС-имитовставкой; при этом в качестве имитовставки используется часть последнего блока в цепочке. Этот режим, к при меру, позволяет проверить, было ли сообщение модифицировано неавторизованной сущностью. Этого можно достичь путем шифрования сообщения с помощью ключа и функции имитовставки. Полученное сообщение можно опять зашифровать с тем же ключом и сравнить имитовставку с той, которую предоставил отправитель. Если они совпадают, это означает, что сообщение не было модифицировано неавторизованным пользователем и, следовательно, целостность была сохранена. В случае несовпадения сообщение было измене но неавторизованной стороной во время передачи.

Режим криптографического хеша

Хеш-функции в основном используются для сжатия сообщений до определен ной длины. В режиме криптографического хеша в качестве функции сжатия используются блочные шифры, которые генерируют хеш исходного текста. На этом мы завершаем знакомство с блочными шифрами. В следующем раз деле мы рассмотрим структуру и принцип работы блочного шифра AES, который в настоящее время доминирует на рынке. Но сначала мы поговорим об истории создания стандарта шифрования данных (англ. Data Encryption Standard, или DES), который привел к появлению нового стандарта, AES.

Стандарт шифрования данных (DES) Стандарт шифрования данных был представлен Национальным институтом стандартов и технологий США в качестве стандартного алгоритма для шифрования и имел широкое распространение в 1980-х и 1990-х годах. Однако из-за усовершенствования технологий и криптографических исследований он не проявил достаточной устойчивости к атакам методом перебора. Например, в июне 1998 года организация Electronic Frontier Foundation (EFF) взломала DES с помощью специального устройства под названием EFF DES cracker (или Deep Crack). DES использует ключ длиной всего 56 бит, что вызывает определенные опасения. Эту проблему должно было решить введение нового стандарта, Triple DES

(3DES), в котором предлагалось использовать ключ длиной 168 бита, состоящий из трех 56-битных ключей; при этом алгоритм DES выполнялся три раза, что делало атаку методом перебора практически бесполезной. Однако это приводило к другим нежелательным ограничениям, связанным с производительностью и размером блока в 64 бита.

Стандарт шифрования AES

В 2001 году проводился открытый конкурс по выбору нового стандарта AES (англ. Advanced Encryption Standard – продвинутый стандарт шифрования), в котором победил алгоритм шифрования Rijndael авторства Йоан Даймен и Винсента Рэймена с минимальными изменениями со стороны NIST. Пока что не обнаружено ни одной атаки на AES, которая была бы эффективней простого перебора. Оригинальная версия Rijndael допускает разные размеры ключей и блоков: 128, 192 и 256 бит. И хотя в стандарте AES поддерживаются только 128-битные блоки, ключи по-прежнему могут быть 128-, 192- и 256-битными.

Как работает AES

Во время своей работы алгоритм AES в несколько проходов модифицирует массив размером 4×4 , который называют состоянием. Для полного шифрования требуется от 10 до 14 итераций, в зависимости от размера ключа. Соответствие между размером ключа и необходимым количеством итераций показано в табл. 3.1.

Таблица 3.1. Соответствие между размером ключа и необходимым количеством итераций

Размер ключа	Количество необходимых итераций
128 бит	10 итераций
192 бита	12 итераций
256 бит	14 итераций

Как только шифр получает инициализированное состояние, выполняется четыре операции в четыре этапа, которые шифруют ввод. Эти этапы называются AddRoundKey, SubBytes, ShiftRows и MixColumns.

- 1. На этапе AddRoundKey к массиву состояния применяется исключающее ИЛИ с подключом, полученным из основного ключа.
 - 2. SubBytes – это этап подстановки, на котором заменяются все байты массива состояния с использованием поисковой таблицы (блока подстановок).
 - 3. На этапе ShiftRows каждый ряд в массиве состояния (кроме первого) сдвигается влево. Это делается циклически и пошагово.
 - 4. В конце, на этапе MixColumns, все байты перемешиваются линейно между разными столбцами.
- Шаги, перечисленные выше, описывают одну итерацию AES. На последней итерации (10-й, 12-й или 14-й, в зависимости от размера ключа) вместо четвертого этапа используется AddRoundKey; благодаря этому первые три этапа не могут быть обращены вспять (рис. 53).

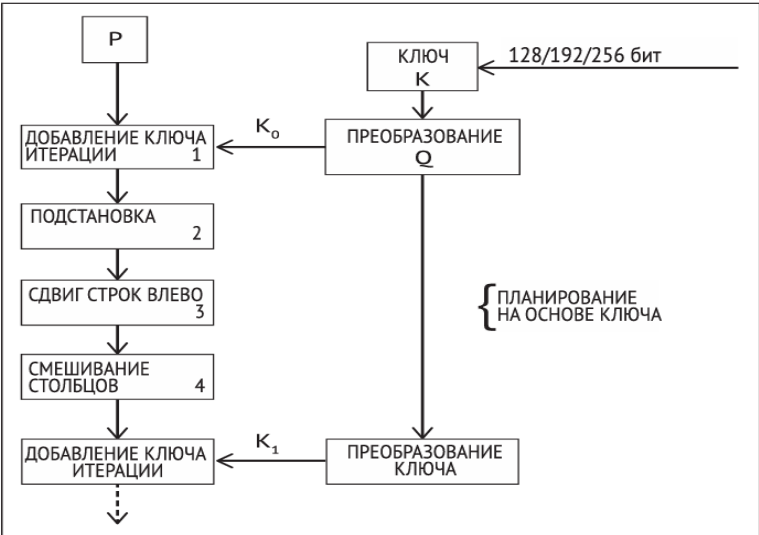


Рис 53 - итерацию AES

Блок-схема первой итерации шифрования методом AES. В последней итерации этап смешивания пропускается. Различные криптовалютные кошельки используют AES для шифрования данных, хранящихся локально. В частности, в кошельке Bitcoin применяется AES-256 в режиме CBC. Ниже показан пример шифрования и дешифрования методом AES в OpenSSL: \$ openssl enc -aes-256-cbc -in message.txt -out message.bin enter aes-256-cbc encryption password: Verifying - enter aes-256-cbc encryption password: \$ ls -ltr-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 05:54 message.txt-rw-rw-r-- 1 drequinox drequinox 32 Sep 21 05:57 message.bin \$ cat message.bin

Вот как выглядит содержимое файла message.bin (рис. 54):

```
Salted__w_____s_ŷ_____h~?_____:/Crypt$
:/Crypt$
```

Рис 54 - Содержимое файла message.bin

Как видите, файл message.bin является двоичным. Иногда для совместимости двоичные файлы кодируют в текстовом формате. Для этого можно использовать следующую команду:

```
$ openssl enc -base64 -in message.bin -out message.b64 $ ls -ltr-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 05:54 message.txt-rw-rw-r-- 1 drequinox drequinox 32 Sep 21 05:57 message.bin-rw-rw-r-- 1 drequinox drequinox 45 Sep 21 06:00 message.b64 $ cat message.b64
U2FsdGVkX193uByIcwZf0Z7J1at+4L+Fj8/uzEDAtJE=
```

Для расшифровки файла, зашифрованного методом AES, можно применить команду, представленную ниже. Здесь используется файл message.bin из предыдущего примера:

```
$ openssl enc -d -aes-256-cbc -in message.bin -out message.dec enter aes-256-cbc decryption password: $ ls -ltr-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 05:54 message.txt-rw-rw-r-- 1 drequinox drequinox 32 Sep 21 05:57 message.bin-rw-rw-r-- 1 drequinox drequinox 45 Sep 21 06:00 message.b64-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 06:06 message.dec $ cat message.dec Datatoencryp
```

Внимательные читатели могли заметить отсутствие вектора IV, хотя он требуется во всех режимах блочного шифрования, кроме ECB. Дело в том, что OpenSSL автоматически извлекает IV из предоставленного пароля. Пользователь может указать собственный вектор с помощью следующего параметра:

-K/-iv , (Initialization Vector) should be provided in Hex.

Команда, показанная ниже, декодирует содержимое файла message.b64 из base64. \$ openssl enc -d -base64 -in message.b64 -out message.ptx \$ ls -ltr-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 05:54 message.txt-rw-rw-r-- 1 drequinox drequinox 32 Sep 21 05:57 message.bin-rw-rw-r-- 1 drequinox drequinox 45 Sep 21 06:00 message.b64-rw-rw-r-- 1 drequinox drequinox 14 Sep 21 06:06 message.dec-rw-rw-r-- 1 drequinox drequinox 32 Sep 21 06:16 message.ptx \$ cat message.ptx

Вот что находится внутри файла message.ptx (рис. 55):

```
:/Crypt$ cat message.ptx
Salted__w_____s_ŷ_____h~?_____:/Crypt$
```

Содержимое файла message.ptx

OpenSSL поддерживает множество разных шифров, список которых приводится на рис. 56. Вы можете исследовать их, отталкиваясь от предыдущих примеров.

Cipher Types		
-aes-128-cbc	-aes-128-ccm	-aes-128-cfb
-aes-128-cfb1	-aes-128-cfb8	-aes-128-ctr
-aes-128-ecb	-aes-128-ofb	-aes-192-cbc
-aes-192-ccm	-aes-192-cfb	-aes-192-cfb1
-aes-192-cfb8	-aes-192-ctr	-aes-192-ecb
-aes-192-ofb	-aes-256-cbc	-aes-256-ccm
-aes-256-cfb	-aes-256-cfb1	-aes-256-cfb8
-aes-256-ctr	-aes-256-ecb	-aes-256-ofb
-aes128	-aes192	-aes256
-bf	-bf-cbc	-bf-cfb
-bf-ecb	-bf-ofb	-blowfish
-camellia-128-cbc	-camellia-128-cfb	-camellia-128-cfb1
-camellia-128-cfb8	-camellia-128-ecb	-camellia-128-ofb
-camellia-192-cbc	-camellia-192-cfb	-camellia-192-cfb1
-camellia-192-cfb8	-camellia-192-ecb	-camellia-192-ofb
-camellia-256-cbc	-camellia-256-cfb	-camellia-256-cfb1
-camellia-256-cfb8	-camellia-256-ecb	-camellia-256-ofb
-camellia128	-camellia192	-camellia256
-cast	-cast-cbc	-cast-cfb
-cast5-cfb	-cast5-ecb	-cast5-ofb
-des	-des-cbc	-des-cfb
-des-cfb1	-des-cfb8	-des-ecb
-des-ede	-des-ede-cbc	-des-ede-cfb
-des-ede-ofb	-des-ede3	-des-ede3-cbc
-des-ede3-cfb	-des-ede3-cfb1	-des-ede3-cfb8
-des-ede3-ofb	-des-ofb	-des3
-desx	-desx-cbc	-id-aes128-CCM
-id-aes128-wrap	-id-aes192-CCM	-id-aes192-wrap
-id-aes256-CCM	-id-aes256-wrap	-id-smime-alg-CMS3DESwrap
-idea	-idea-cbc	-idea-cfb
-idea-ecb	-idea-ofb	-rc2
-rc2-40-cbc	-rc2-64-cbc	-rc2-cbc
-rc2-cfb	-rc2-ecb	-rc2-ofb
-rc4	-rc4-40	-seed
-seed-cbc	-seed-cfb	-seed-ecb
-seed-ofb		

Рис 56 - Снимок экрана с богатым выбором параметров в OpenSSL

С помощью утилиты OpenSSL можно поэкспериментировать со всеми шифрами, представленными на этом снимке.

Асимметричное шифрование

Асимметричное шифрование – это вид криптографии, в котором ключ шифрования отличается от ключа дешифрования. Такое шифрование также называют шифрованием с открытым ключом. При этом подходе для шифрования и дешифрования данных используются открытый и закрытый ключи соответственно. На практике применяется множество схем асимметричного шифрования, в том числе RSA, DSA и ElGamal. Общий принцип шифрования с открытым ключом показан на рис. 57. Рисунок 57 показывает, как отправитель шифрует текст Т с помощью открытого ключа получателя и функции шифрования Ш. Получившиеся зашифрованные данные З передаются получателю по сети. Как только данные достигнут получателя, их можно будет расшифровать с помощью закрытого ключа получателя. При этом зашифрованные данные З будут обработаны функцией Д, на выходе которой получатся данные в виде текста Т. Таким образом, закрытый ключ остается у получателя (в отличие от систем симметричного шифрования, где отправителю и получателю необходим общий ключ, для того чтобы шифровать и дешифровать данные).

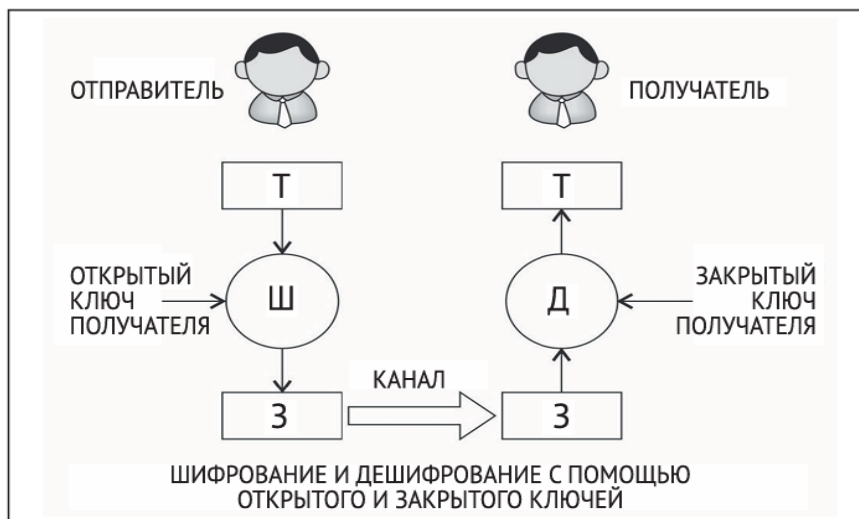


Рис 57 - схема асимметричного шифрования

Шифрование и дешифрование с помощью открытого и закрытого ключей

Рисунок 4.2 показывает, как получатель использует шифрование с открытым ключом, чтобы проверить целостность полученных данных. В этой модели отправитель подписывает данные своим закрытым ключом, после чего отправляет их получателю. Как только данные получены, их целостность проверяется с помощью открытого ключа отправителя. Обратите внимание, что в этой модели не происходит шифрования данных. Задача этой схемы – дать вам представление о процессах аутентификации и валидации данных. Эти процессы будут более подробно описаны в соответствующих разделах данной главы.

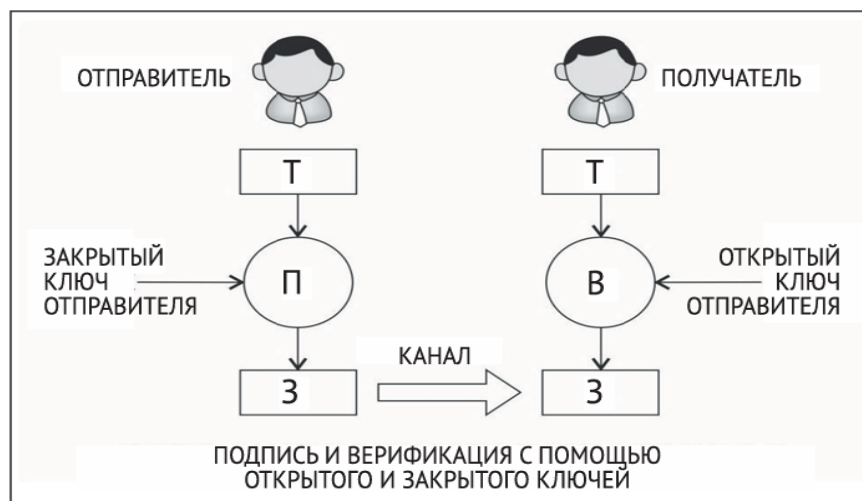


Рис 58 - Модель механизма подписи при шифровании с открытым ключом

Как показано на рис. 4.2, отправитель ставит цифровую подпись на текстовые данные Т, используя свой закрытый ключ и функцию подписи П. Получившиеся зашифрованные данные З затем передаются получателю, который проверяет их с помощью открытого ключа отправителя и функции В. Таким образом получатель удостоверяется, что данные были переданы ему именно этим отправителем. В системах шифрования с открытым ключом содержатся такие механизмы безопасности, как обмен ключами, цифровые подписи, идентификация, шифрование и дешифрование данных. Механизмы обмена ключами связаны с проблемой разработки протоколов, позволяющих создавать ключи для передачи данных по незащищенному каналу. Неотказуемость, необходимая во многих случаях, может быть обеспечена с помощью цифровых подписей. В некоторых случаях важно не только удостоверить личность пользователя, но и идентифицировать сущность, участвующую в транзакции. Этого можно добиться, одновременно используя цифровые подписи и протоколы двусторонней аутентификации. Наконец, механизм шифрования для защиты конфиденциальной информации также может быть создан с помощью систем шифрования с открытым ключом (на пример, RSA, ECC или ElGamal).

Обработка данных с использованием алгоритмов асимметричного шифрования занимает больше времени, чем при симметричном шифровании. По этому асимметричное шифрование редко используется для больших файлов или для шифрования самих данных. Чаще такое шифрование используется для обмена ключами в рамках алгоритма симметричного шифрования. Таким образом, можно осуществить безопасный обмен ключами с помощью асимметричного шифрования, а затем применять алгоритмы симметричного шифрования для непосредственной обработки данных. Алгоритмы шифрования с открытым ключом базируются на различных математических функциях. Рассмотрим три основные категории алгоритмов асимметричного шифрования.

Целочисленная факторизация Алгоритмы целочисленной факторизации основаны на том, что большие целые числа очень сложно факторизовать. Ярким примером такого алгоритма является RSA. **Дискретное логарифмирование** Алгоритм на основе дискретного логарифмирования базируется на задаче модульной арифметики. Вычислить остаток от деления несложно; при этом поиск показателя степени первообразного корня – очень трудоемкая задача. Иными словами, крайне проблематично найти входные данные, зная лишь результат. Это односторонняя функция.

Функции хеширования

Функции хеширования используются для того, чтобы представлять данные произвольного размера в краткой форме фиксированной длины. Функции хеширования не имеют ключа и защищают целостность данных. Как правило, они создаются с помощью специальных итерированных техник построения функций хеширования. Существуют различные группы функций хеширования, такие как MD, SHA-1, SHA-2, SHA-3, RIPEMD и Whirlpool. Функции хеширования часто используются для цифровых подписей и имитовставок (Message Authentication Codes, MACs) – например, HMAC. У них есть три характеристики безопасности: вычислительная неразрешимость, вторичная вычислительная неразрешимость и устойчивость к коллизиям. Мы рассмотрим эти характеристики

далее в этом разделе. Кроме того, функции хеширования часто применяются для обеспечения целостности данных. Они могут использоваться и как односторонние функции, и как основа для создания других базовых криптографических элементов (например, имитовставок и цифровых подписей). Некоторые приложения используют функции хеширования в качестве генератора псевдослучайных чисел (Pseudo-random Numbers Generator, PRNG). В следующих подразделах мы рассмотрим две практические характеристики и три характеристики безопасности, присущие функциям хеширования.

Сжатие данных случайной длины и представление их в форме установленного размера Функция хеширования должна быть способна обработать текст любой длины и сжать его до сообщения заданного размера. Размер сообщения, полученного в результате хеширования, чаще всего установлен в промежутке от 128 до 512 бит. Простота вычислений Функции хеширования являются односторонними. Они отличаются эффективностью и скоростью выполнения. Обязательным условием является то, что функция хеширования должна выполняться очень быстро вне зависимости от размера сообщения. Если сообщение имеет слишком большой размер, эффективность функции хеширования может падать; тем не менее даже в этом случае хеширование должно выполняться с достаточно высокой скоростью для практического применения. Теперь рассмотрим характеристики безопасности функции хеширования.

Вычислительная неразрешимость Эта характеристика может быть объяснена простым уравнением: $h(x) = y$. Здесь h – функция хеширования, x – входные данные, а y – хеш. Первой характеристикой безопасности является то, что x невозможно рассчитать на основе y . x считается прообразом y , поэтому данная характеристика носит название вычислительная неразрешимость. Иными словами, функция хеширования должна быть односторонней. Вторичная вычислительная неразрешимость Согласно требованиям вторичной вычислительной неразрешимости, при данных x и $h(x)$ невозможно найти такое сообщение m , чтобы $m \neq x$, а $h(m) = h(x)$ (то есть хеш m не должен совпадать с хешем x). Эта характеристика также называется слабой устойчивостью к коллизиям.

Устойчивость к коллизиям Устойчивость к коллизиям требует, чтобы два различных входных сообщения не могли привести к получению одинакового хеша на выходе. Эту характеристику (также называемую сильной устойчивостью к коллизиям) можно представить неравенством $h(x) \neq h(z)$. Все перечисленные характеристики отражены на рис. 59.

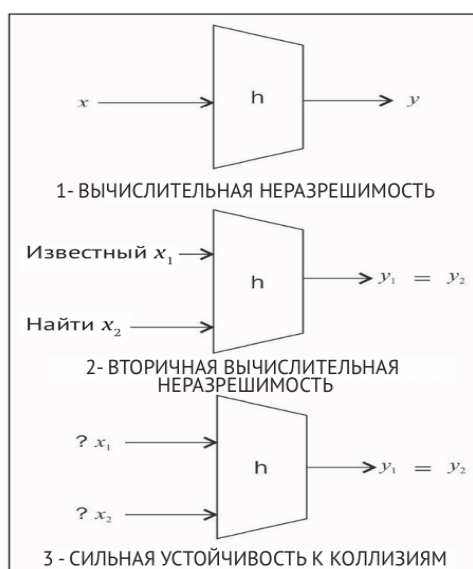


Рис 59 - Три характеристики безопасности функций хеширования

В силу своей природы функции хеширования неизбежно имеют какое-то количество коллизий, то есть хеш двух разных сообщений так или иначе может быть одинаковым. Однако вычислить их должно быть слишком сложно. В любых функциях хеширования приветствуется особенность, которая называется лавинным эффектом. Благодаря лавинному эффекту любое малейшее изменение входного текста (даже на один символ) приводит к совершенно другому хешу.

Как правило, при создании функций хеширования применяется концепция итерированных хеш-функций. При таком подходе происходит поочередное многоэтапное сжатие блоков входного сообщения, а на выходе получаются данные в сжатом виде. Распространенный вид итерированных функций хеширования – структура Меркла–Дамгарда. В основе этой структуры лежит идея разделения входных данных на блоки одинакового размера, которые затем последовательно обрабатываются функциями сжатия. Устойчивость к коллизиям, присущая функциям сжатия, гарантирует, что полученный хеш также будет устойчив к коллизиям. Функции хеширования могут быть построены с помощью блочных шифров. Помимо структуры Меркла–Дамгарда, существуют и другие структуры функций сжатия: например, структура Миагучи–Пренеля и структура Девиса–Мейера.

В следующих подразделах мы рассмотрим различные категории функций хеширования.

Резюме сообщения

Резюмирующие функции (Message Digest, MD) были широко распространены в начале 90-х годов. В эту категорию попадают функции MD4 и MD5. В обеих функциях были обнаружены уязвимости, и сейчас они считаются небезопасными. MD5 – это 128-битная функция хеширования, которая часто применялась для проверки целостности файлов.

Применение криптографических мер безопасности для защиты транзакций в блокчейне

Алгоритмы безопасного хеширования

Ниже вы видите список самых популярных алгоритмов безопасного хеширования (Secure Hash Algorithms, SHA):

SHA-0: эта 160-битная функция была предложена NIST в 1993 году;

SHA-1: эта функция была предложена NIST в 1995 году в качестве замены SHA-0. SHA-1 также является 160-битной хеш-функцией и широко применяется в SSL и TLS. Необходимо отметить, что эта функция теперь считается небезопасной и не одобряется сертифицирующими органами. В новых разработках не принято использовать SHA-1;

SHA-2: эта группа функций включает в себя четыре функции (SHA 224, SHA-256, SHA-384 и SHA-512), различающиеся размером хеша в битах; SHA-3: новейшее поколение функций SHA, состоящее из функций SHA 3-224, SHA-3-256, SHA-3-384 и SHA-3-512.

SHA-3 – это версия Кессак, утвержденная NIST в качестве стандарта. Кессак использует новый метод криптографической губки вместо широко распространенного преобразования Меркла–Дамгарда;

RIPEMD: это название расшифровывается как RACE Integrity Primitives Evaluation Message Digest (резюмирующая функция оценки примитивов целостности RACE). Алгоритм основан на тех же принципах, что и MD4. Существуют различные версии RIPEMD, в том числе 128-битная, 160-битная, 256-битная и 320-битная;

Whirlpool: этот алгоритм основан на модифицированной версии шифра «Рэндал» (Rijndael), известной как W. В нем используется функция сжатия Миагучи–Пренеля – односторонняя функция, позволяющая сжать два входных сообщения фиксированной длины до одного выходного сообщения фиксированной длины.

На практике функции хеширования применяются в самых разных целях: от простой проверки целостности файла и хранения паролей до использования в протоколах и алгоритмах шифрования. Они используются в хеш-таблицах, распределенных хеш-таблицах, фильтрах Блума, а также для обнаружения сигнатур вирусов, обмена данными в пиринговых сетях и выполнения многих других задач.

Функции хеширования играют важнейшую роль в блокчейне. В частности, функция доказательства выполнения работы (PoW) дважды использует SHA 256, чтобы оценить объем вычислительных ресурсов, затраченных майнерами. Для генерации Bitcoin-адресов используется RIPEMD 160; об этом мы поговорим подробнее в следующих главах. Теперь рассмотрим устройство алгоритма SHA.

Устройство алгоритмов безопасного хеширования (SHA)

В этом подразделе мы рассмотрим устройство алгоритмов SHA-256 и SHA-3, использующихся в системах Bitcoin и Ethereum соответственно. Ethereum использует Кессак (оригинальный алгоритм, представленный на рассмотрение NIST) вместо стандартного алгоритма SHA-3, утвержденного NIST. После не которых модификаций (таких как увеличение количества циклов и упрощение процесса дополнения сообщений) NIST утвердил Кессак в качестве стандарта SHA-3.

Устройство SHA-256

Длина входного сообщения в SHA-256 должна быть меньше 264 бит. Размер блока – 512 бит, а размер слова – 32 бит. Выходное сообщение имеет длину 256 бит. Функция сжатия обрабатывает 512-битный блок сообщения и 256-битное промежуточное значение хеша. Эта функция состоит из двух основных компонентов: функции сжатия и распорядка сообщения. Алгоритм состоит из восьми шагов.

1. Подготовка

1. Осуществляется дополнение сообщения, чтобы добиться длины блока в 512 бит (в случае если размер блока меньше 512 бит).
2. Дополненное сообщение разбирается на отдельные блоки, размер каждого из которых равен 512 бит.
3. Устанавливается изначальное значение хеш-функции, состоящее из восьми 32-битных слов, полученных путем взятия первых 32 бит дробных частей квадратных корней первых восьми простых чисел. 2. Расчет хеш-суммы
4. Каждый блок обрабатывается поочередно. Полный расчет хеш-суммы производится в 64 цикла. В каждом цикле используются слегка отличающиеся от других циклов константы. Таким образом, результат рас чета каждого цикла уникален.
5. Создается распорядок сообщения.
6. Задаются восемь вспомогательных переменных.
7. Рассчитывается промежуточная хеш-сумма.
8. Наконец, производится обработка сообщения, в результате чего полу чается итоговая хеш-сумма.

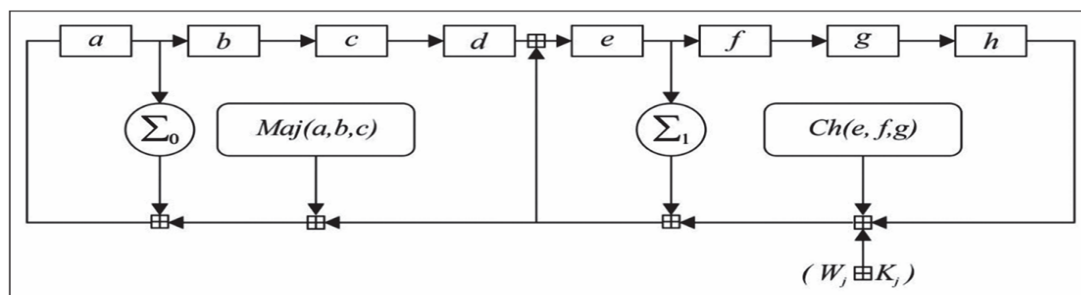


Рис 60 - Один цикл функции сжатия SHA-256

На рис. 60 a, b, c, d, e, f, g и h – это вспомогательные переменные. Maj и Ch применяются поразрядно. Σ_0 и Σ_1 осуществляют битовый сдвиг. Константами цикла являются W_j и K_j , которые складываются по модулю 232. Устройство SHA-3 (Кессак) Структура SHA-3 сильно отличается от SHA-1 и SHA-2. В основе SHA-3 лежат псевдослучайные перестановки, тогда как перестановки в других хеш-функциях обычно не случайны. Кроме того, Кессак не использует преобразование Меркла–Дамгарда, широко применяющееся в хеш-функциях для обработки данных произвольной длины. Вместо этого Кессак использует более новый подход, который носит название криптографической губки. Это модель случайных перестановок. В группу SHA-3 входит множество стандартов: SHA-3 224, SHA-3-256, SHA-3-384, SHA-3-512, SHAKE-128 и SHAKE-256. SHAKE-128 и SHAKE-256 – это «функции с удлиняемым результатом» (Extendable Output Functions, XOF),

также утвержденные NIST в качестве стандарта. Такие функции позволяют удлинять результат до любого требуемого размера.

На рис. 61 представлена модель криптографической губки, лежащая в основе SHA-3 и Кеска. Название модели выбрано не случайно: дополненное сообщение сначала «впитывается в губку», а затем преобразуется в подмножество состояний перестановки с помощью XOR. После этого преобразованные данные «выжимаются» из губки. Размер входного блока функции губки называется «скоростью», а общий уровень безопасности определяется «емкостью» функции:

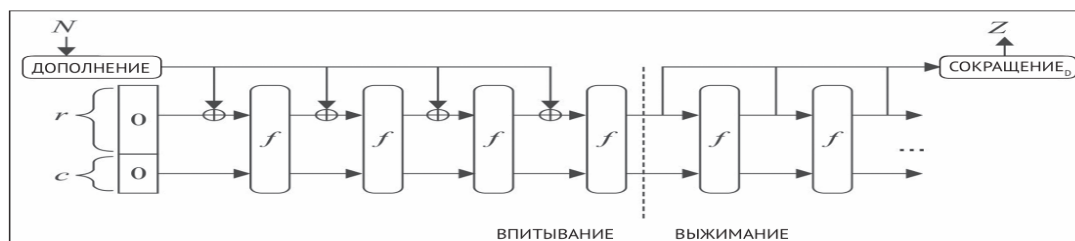


Рис 61 - Функция губки SHA-3

Пример функций хеширования в OpenSSL СОКРАЩЕНИЕ D Вычислим 256-битную хеш-сумму сообщения «Hello» с помощью алгоритма SHA-256. Для этого воспользуемся следующей командой:

```
$ echo -n 'Hello' | openssl dgst -sha256 (stdin)=
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Обратите внимание, что малейшее изменение исходного текста (даже смена регистра буквы Н) приведет к серьезным изменениям в хеш-сумме. Это и есть лавинный эффект, о котором мы говорили чуть раньше:

```
$ echo -n 'hello' | openssl dgst -sha256 (stdin)=
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

Сравним два результата и убедимся в том, что они совершенно различны:

```
Hello: 18:5f:8d:b3:22:71:fe:25:f5:61:a6:fc:93:8b:2e:26:43:06:ec:30:4e:da:51:80:07:d1:76:48:26:38:19:69
hello: 2c:f2:4d:ba:5f:b0:a3:0e:26:e8:3b:2a:c5:b9:e2:9e:1b:16:1e:5c:1f:a7:42:5e:73:04:33:62:93:8b:98:24
```

Как правило, функции хеширования не используют ключ. Если же использовать эти функции с ключом, то их можно применять для построения структуры шифрования, известной как имитовставка. Коды аутентификации сообщений (MAC-коды, имитовставки) Имитовставки (которые также известны как хеш-функции с ключом) используются для обеспечения целостности и аутентификации сообщений. Точнее говоря, они обеспечивают аутентификацию источника данных. Имитовставки – это симметричные криптографические примитивы, подразумевающие наличие общего ключа у отправителя и получателя. Имитовставки могут быть созданы с использованием блочных шифров или функций хеширования.

Имитовставки на основе блочных шифров При этом подходе имитовставки создаются на основании блочных шифров в режиме сцепления блоков шифротекста (Cipher Block Chaining, CBC). Для этого может использоваться любой блочный шифр – например, AES в режиме CBC. Имитовставка сообщения – это результат последнего цикла операции CBC. Длина имитовставки равна длине блока в блочном шифре, использованном для генерации этой имитовставки. Для верификации вычисляется имитовставка сообщения, которая затем сравнивается с полученной имитовставкой. Если имитовставки совпадают, целостность сообщения считается подтвержденной, в противном случае – нарушенной. Необходимо также отметить, что имитовставки могут работать как цифровые подписи, но не могут обеспечить неотказуемость в силу своей симметричной природы. Имитовставки на основе функций хеширования Как и сами хеш-функции, имитовставки на их основе (Hash-based MACs, HMACs) преобразуют входное сообщение произвольной длины в строку заданной длины. При этом подходе отправитель подписывает сообщение с помощью имитовставки, а получатель проверяет ее с помощью общего ключа. Ключ хешируется вместе с сообщением; для этого используется один из двух методов: секретный префикс либо секретный суффикс. В первом случае ключ сцепляется с сообщением таким образом, что идет перед сообщением. Если же используется метод секретного суффикса, то ключ идет после сообщения. Эти методы можно представить в виде следующих уравнений: Секретный префикс: $M = \text{MAC}_k(x) = h(k \parallel x)$; Секретный суффикс: $M = \text{MAC}_k(x) = h(x \parallel k)$. Оба метода имеют собственные преимущества и недостатки. Известны случаи атак на оба метода. Исследователями в области криптографии были предложены

различные НМАС-структуры, в том числе *ipad* (внутреннее дополнение) и *opad* (внешнее дополнение). Такие структуры считаются безопасными с некоторыми допущениями (рис. 62).

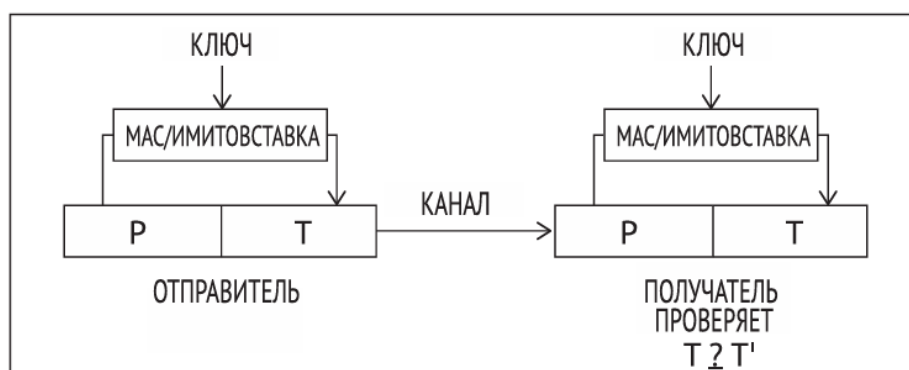


Рис 62 - Работа функции имитовставки

Хеш-функции занимают важное место в работе пиринговых сетей и в блокчейн-технологиях. В следующих подразделах мы рассмотрим некоторые значимые примеры: деревья Меркла, деревья Patricia (сжатые префиксные деревья) и распределенные хеш-таблицы (DHT). Деревья Меркла На рис. 63 представлена схема, предложенная Ральфом Мерклом и получившая название «деревя Меркла». Деревья Меркла обеспечивают быструю и безопасную верификацию больших массивов данных.

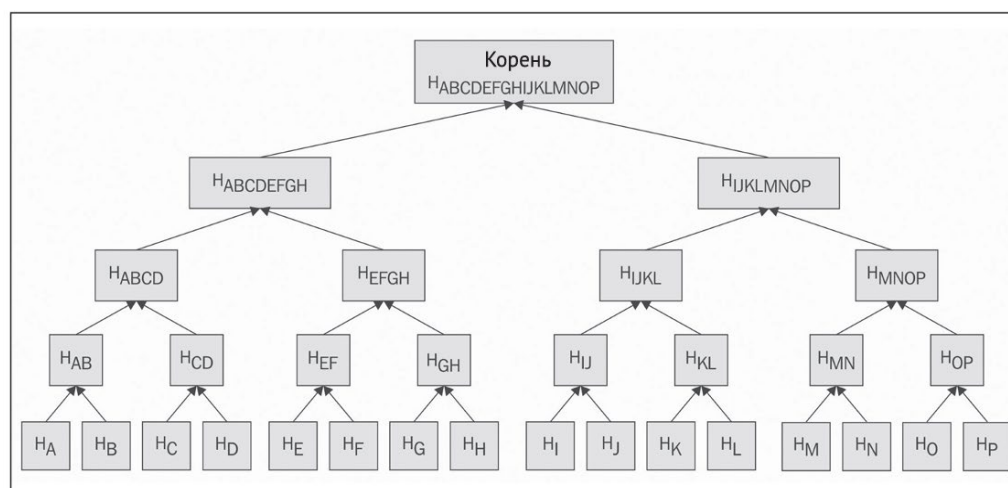


Рис 63 - Дерево Меркла

Дерево Меркла

Дерево Меркла – это бинарное дерево, в котором входные данные сначала помещаются в листья (узлы без потомков). Затем значения каждой пары потомков хешируются вместе, а полученное значение записывается в соответствующий родительский узел (внутренний узел). Этот процесс повторяется до тех пор, пока не будет получено единое значение хеша, известное как корень дерева Меркла.

Цифровые подписи

Цифровые подписи позволяют установить связь между сообщением и сущностью, создавшей это сообщение. Цифровые подписи используются для аутентификации источника данных и обеспечения неотказуемости.

Цифровые подписи находят применение на блокчейне, где отправитель подписывает транзакцию своим закрытым ключом, прежде чем отправить ее в сеть. Таким образом отправитель подтверждает, что является законным владельцем передаваемых активов – например, в валюте биткойн. Транзакция затем проверяется другими узлами сети, чтобы удостовериться, что активы действительно принадлежат этому пользователю (узлу). Эти концепции будут более подробно

рассмотрены в главах данной книги, посвященных системам Bitcoin и Ethereum. Цифровые подписи вычисляются в два этапа. В качестве примера рассмотрим высокоуровневые шаги в схеме цифровых подписей RSA.

Алгоритм цифровой подписи RSA

Алгоритм цифровой подписи RSA состоит из следующих шагов: 1. 2. Расчет хеш-суммы пакета данных. Это гарантирует целостность данных, поскольку получатель тоже может вычислить хеш-сумму и сравнить ее с оригинальной хеш-суммой. Если значения совпадают, то данные не были изменены в процессе передачи. С технической точки зрения, подписи могут работать без первоначального хеширования данных, однако этот вариант считается небезопасным. Подпись хеш-суммы закрытым ключом отправителя. Поскольку за закрытый ключ находится только у лица, поставившего подпись, гарантируется аутентичность подписи и данных, на которые она была поставлена. Важными характеристиками цифровой подписи являются аутентичность, невозможность подделки и невозможность повторного использования. Аутентичность подразумевает, что получатель может проверить подлинность цифровой подписи. Невозможность подделки гарантирует, что только отправитель может поставить подпись с помощью своего закрытого ключа. Иными словами, никто, кроме настоящего отправителя, не может создать такое же подписанное сообщение. Невозможность повторного использования подразумевает, что цифровая подпись не может быть отделена от сообщения и использована для заверения другого сообщения. На рис. 64 показана схема работы стандартной функции цифровой подписи.

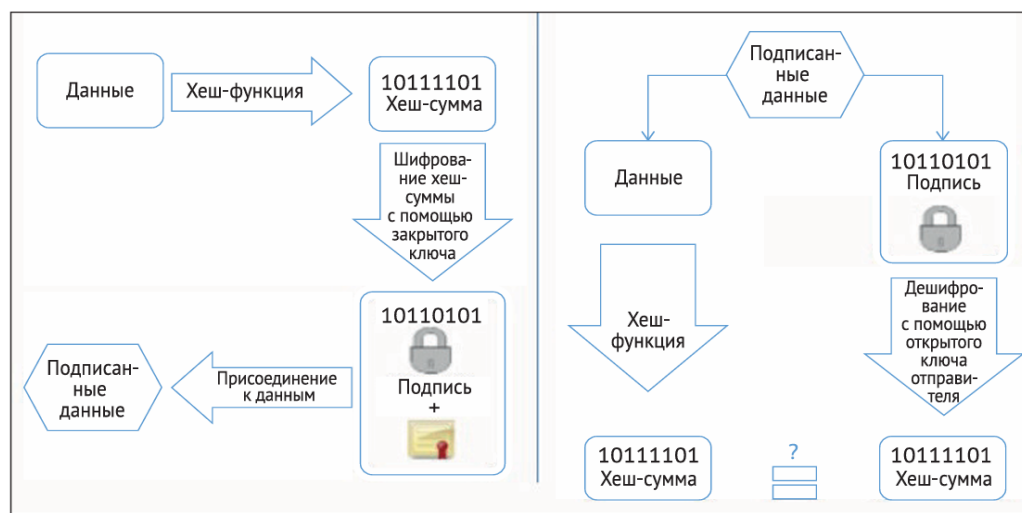


Рис 64 - схема работы стандартной функции цифровой подписи

Цифровая подпись (слева) и процесс верификации (справа) (пример цифровых подписей RSA)

Если отправитель хочет передать получателю аутентифицированное сообщение, то он может сделать это двумя способами: подписать и зашифровать либо зашифровать и подписать. Рассмотрим оба способа подробнее.

Подписать и зашифровать

При этом подходе отправитель ставит на данные цифровую подпись с помощью закрытого ключа, присоединяет подпись к данным, а затем шифрует данные и подпись с помощью открытого ключа получателя. Этот способ считается более безопасным, чем следующий.

Зашифровать и подписать

При этом подходе отправитель сначала шифрует данные с помощью открытого ключа получателя, а затем ставит на зашифрованные данные свою цифровую подпись.

На практике цифровой сертификат, содержащий цифровую подпись, выпускается сертифицирующим органом, который устанавливает связь между открытым ключом и личностью пользователя.

Практическое применение находят такие схемы, как RSA, DSA (Digital Signature Algorithm, Алгоритм цифровой подписи) и схемы на базе ECDSA. Чаще всего используется RSA; в то же время на волне популярности ECC признание получают и схемы на базе ECDSA. Это хорошая новость для блокчейна, поскольку ECC обеспечивает тот же уровень безопасности, что и RSA, занимая при этом меньше места. Кроме того, генерация ключей в ECC осуществляется гораздо быстрее, чем в RSA; таким образом, повышается производительность всей системы. Таблица 4.1 показывает, что ECC позволяет добиться такого же уровня защиты, что и RSA, при меньшем размере ключей.

Алгоритм Signcryption

Алгоритм Signcryption – это примитив шифрования с открытым ключом, объединяющий в себе все функции цифровой подписи и шифрования. Автором Signcryption является Юлян Чжен (Yuliang Zheng). Сейчас система описывается стандартом ISO, ISO/IEC 29150:2011. При традиционном подходе, для обеспечения аутентификации, неотказуемости и невозможности подделки, используются схемы подписать и зашифровать либо зашифровать и подписать. Однако Signcryption предлагает все функции цифровых подписей и шифрования с меньшими издержками, чем у схемы «подписать и зашифровать». Для алгоритма Signcryption верно неравенство Издержки (подпись и шифрование) << Издержки (подпись) + Издержки (шифрование).

Доказательства с нулевым разглашением Доказательства с нулевым разглашением (Zero-Knowledge Proofs, ZKPs) были представлены Гольдвассер, Микали и Рекоффом в 1985 году. Эти доказательства позволяют убедиться в подлинности утверждения без разглашения какой-либо информации о самом утверждении. Доказательства с нулевым разглашением должны обладать тремя свойствами: полнотой, корректностью и нулевым разглашением. Полнота подразумевает, что доказывающий обязательно убедит проверяющего в верности утверждения, если оно действительно верно. Свойство корректности гласит, что доказывающий не может убедить проверяющего в верности утверждения, если утверждение на самом деле ложно. Наконец, нулевое разглашение является главной характеристикой таких доказательств и подразумевает, что об утверждении абсолютно ничего не известно, за исключением того, верно оно или ложно. Доказательства с нулевым разглашением вызвали особый интерес у исследователей в области блокчейн-технологий, поскольку они предлагают характеристики безопасности, предпочтительные для таких сфер деятельности, как финансы, юриспруденция и медицина. Недавний пример успешного применения механизма доказательств с нулевым разглашением – криптовалюта Zcash. В Zcash используется особый тип таких доказательств, получивший название Краткого неинтерактивного аргумента доказательств с нулевым разглашением (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge, ZK-SNARK). Мы рассмотрим эту тему подробнее в главе 8.

Слепые подписи

Слепые подписи были предложены Дэвидом Чаумом в 1982 году. Они основаны на системах цифровой подписи с открытым ключом (например, RSA). Основная идея слепых подписей заключается в том, чтобы заставить подписывающую сторону подписать сообщение, не зная его содержания. Это достигается путем скрытия сообщения. Как и обычная цифровая подпись, слепая подпись может быть верифицирована по оригинальному сообщению. Слепые подписи были разработаны как механизм для развития систем цифровой валюты.

Схемы кодирования

Помимо криптографических примитивов, часто используются схемы кодирования, преобразующие двоичный код в текст. Чаще всего двоичные данные конвертируются в текст для последующей обработки, сохранения и передачи по каналу, не поддерживающему работу с данными в двоичном представлении. Например, графические изображения иногда хранятся в базах данных в популярной кодировке base64, позволяющей сохранять изображения в текстовом поле. Другая схема кодирования – base58 – стала популярной благодаря своему использованию в системе Bitcoin. Криптография – это широкая сфера, и в данном разделе мы рассмотрели лишь базовые понятия, необходимые для ее общего понимания и (особенно) понимания в разрезе блокчейна и криптовалют. В следующем разделе мы познакомимся с базовыми концепциями финансового рынка. Также будет приведена общая терминология, касающаяся торговли, бирж и торговых циклов. Более подробная информация будет дана в последующих главах, где будут рассмотрены конкретные кейсы.